RSCL — Algorithm 9: Setpoint Tracking, Arrival Detection (MOVE/TURN/TAKEOFF to HOVER) and Stale-Telemetry Skipping

Reconfigurable Space Computing Lab (RSCL), California State Polytechnic University, Pomona (Cal Poly Pomona) · Director: Prof. Mohamed El-Hadedy

Date: 2026-11-12 · Document type: formal algorithm specification of the implemented code (HU-MA'AT drone-fleet simulation demo)
Source files described (content hash and state at document build time):

hu-telepresence-gazebo/festival/control_server_hu.py	sha256 3f32578a144c	2026-11-11 06:32 UTC	clean (HEAD 983eede; last change 10ee643)

1. Purpose and context

The supervisor converts mode + setpoint into velocity commands for the Gazebo multicopter velocity controller and decides when a manoeuvre is complete. Completion matters for safety and usability: subsequent single-aircraft commands require HOVER (Algorithm 7.2), group moves require "settled" members (Algorithm 8), and the mission coordinator verifies steps from these modes (Algorithm 11). A former defect (aircraft remaining in MOVE forever, so that every following move was refused) is fixed by the arrival rule described here. The same loop implements the telemetry watchdog that withholds motion from aircraft whose odometry is stale.

2. Inputs, outputs

Inputs per tick: position p, velocity v, yaw ψ (from quaternion), setpoint (x*, y*, z*, ψ*), mode, odometry receive time. **Outputs:** body-frame linear velocity (b_x, b_y), vertical velocity v_z, yaw rate r_z with motor enable; mode transitions and events `HOVER`, `ARRIVED`, `SPIN_COMPLETE`.

3. Algorithm

Algorithm 9.1 Tracking and arrival (per fresh, non-parked, non-held aircraft)

Source: festival/control_server_hu.py: Fleet.loop (after the watchdog of Algorithm 7.1), Fleet.on_odom, Fleet.fresh

```
every 0.10 s for aircraft d (fresh, not parked/queued/faulted, not held this tick) do
  if d.mode = SPIN then
    δψ ← wrap(ψ* − ψ)
    if |δψ| < 0.12 rad then
      d.spin_segments ← d.spin_segments − 1
      if d.spin_segments ≤ 0 then d.mode ← HOVER; ψ* ← ψ; emit SPIN_COMPLETE
      else ψ* ← wrap(ψ* + dir·π/2)                          // next 90° segment
  e ← (x* − x, y* − y, z* − z)
  vx ← clamp(0.80·e_x − 0.16·v_x, ±0.62);  vy ← clamp(0.80·e_y − 0.16·v_y, ±0.62)
  vz ← clamp(1.05·e_z − 0.22·v_z, ±0.45);  if d.mode = LANDING then vz ← clamp(vz, −0.32, +0.10)
  (b_x, b_y) ← ( cosψ·vx + sinψ·vy,  −sinψ·vx + cosψ·vy )   // world → body (controller expects body frame)
  e_ψ ← wrap(ψ* − ψ);  r_z ← clamp(1.25·e_ψ, ±0.55)
  publish(enable, b_x, b_y, vz, r_z)
  if d.mode = TAKEOFF and |e_z| < 0.13 and |v_z| < 0.18 then d.mode ← HOVER; emit HOVER
  else if d.mode ∈ {MOVE, TURN} and ‖e‖ < 0.06 and ‖v‖ < 0.08 and |e_ψ| < 0.05 then
    d.mode ← HOVER; emit ARRIVED                            // FORMATION has no automatic transition
```

Algorithm 9.2 Freshness and stale-telemetry skipping

Source: festival/control_server_hu.py: Fleet.fresh, Fleet.loop, Fleet.command

```
fresh(d) ≡ d.init and (now − d.last_seen) < 2.0 s          // last_seen set on every odometry message
in the loop: if not fresh(d) then publish zero velocity without enable; skip d
in a command: stale ← { n ∈ targets : not fresh(n) }
  if stale ≠ ∅ then
```

```
    if action ∈ {land, hover} then targets ← targets \ stale; emit SAFETY_SKIPPED(stale)   // safe direction
    else refuse "No fresh odometry for ..."                                                 // fail closed
  formation additionally requires all five aircraft fresh (Algorithm 5)
on first odometry of d: setpoint ← current pose; mode ← READY
```

4. Flowchart and state diagram

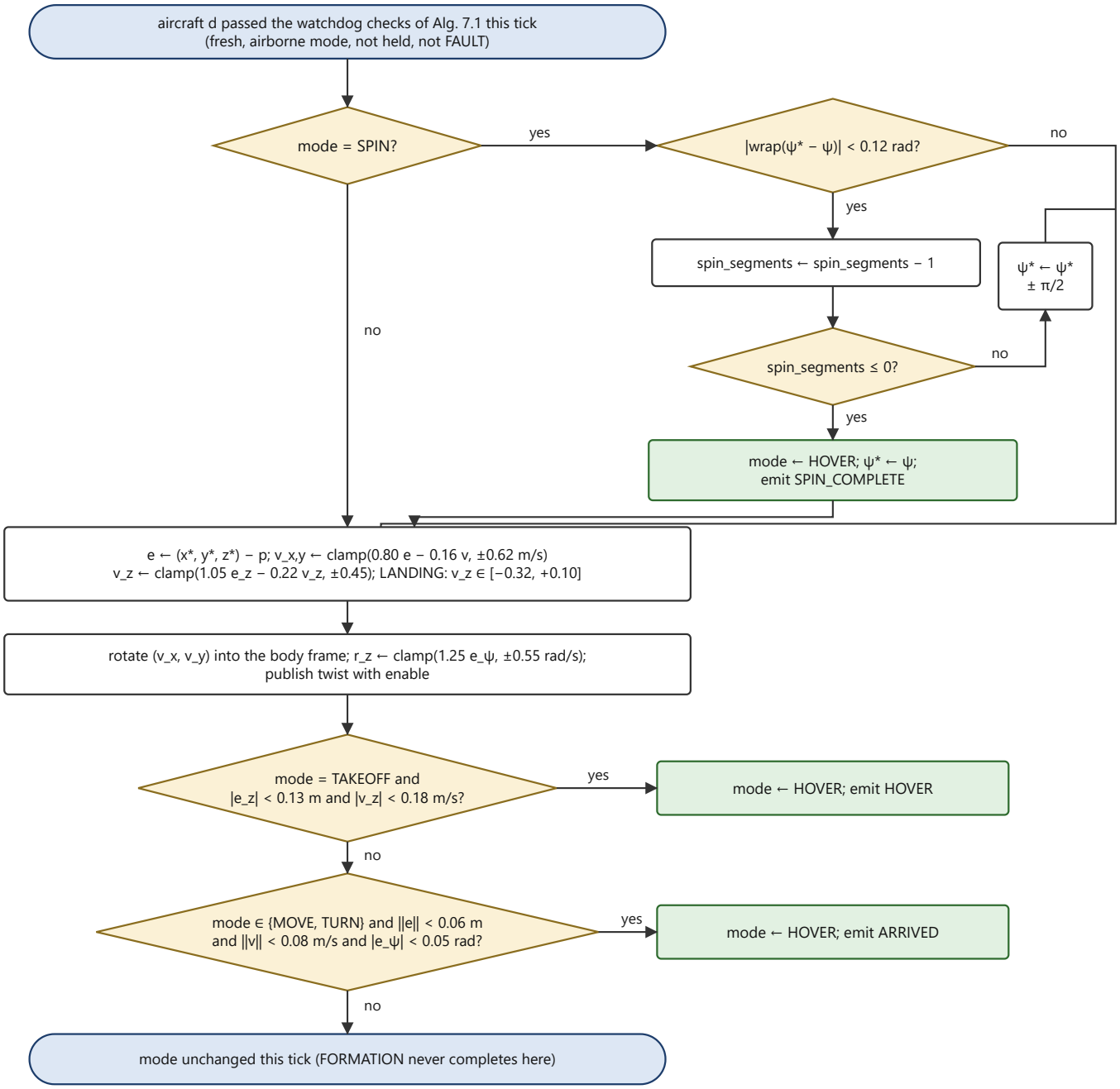


Figure 1. Flowchart of setpoint tracking and arrival detection in `Fleet.loop`, run for an aircraft after the watchdog part (Algorithm 7, Fig. 1) did not end its tick. All thresholds are single-tick tests.

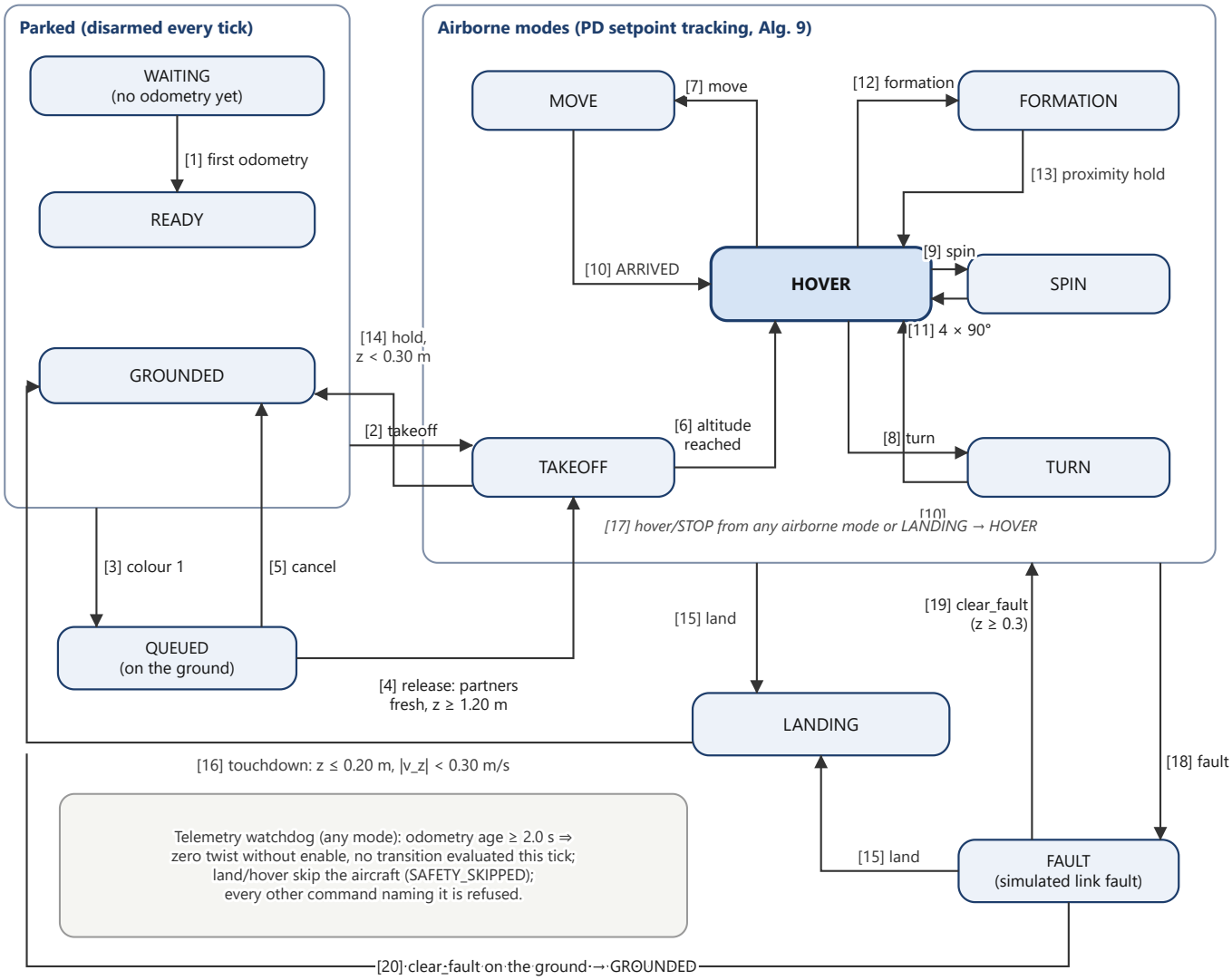


Figure 2. Complete per-drone mode FSM of the MA'AT flight supervisor (`Drone.mode`). Bracketed numbers refer to the table below. "Parked" aircraft are actively disarmed; QUEUED is parked for the watchdog but receives no command.

Guard table for Fig. 2 (all transitions of `control_server_hu.py`; command-time guards apply only after the common checks: target fault refusal, stale fail-closed except land/hover):

#	Transition	Guard / trigger (as implemented)	Where
1	WAITING → READY	first odometry message; setpoint ← current pose	on_odom
2	READY/GROUNDED/WAITING → TAKEOFF	takeoff, fresh, no fault; no close parked partner or colour 0 (upper layer)	command, _plan_stagger
3	parked → QUEUED	takeoff, colour 1 of a close pair (h < 0.88 m); target layer = base	command
4	QUEUED → TAKEOFF	every awaited partner fresh, z ≥ 1.20 m, not faulted (STAGGER_RELEASE)	loop
5	QUEUED → GROUNDED	> 25 s waiting, or a partner faulted / LANDING / GROUNDED (STAGGER_CANCELLED); hover or land command	loop, command
6	TAKEOFF → HOVER	$\|e_z\| < 0.13$ m and $\|v_z\| < 0.18$ m/s	loop
7, 8, 9	HOVER → MOVE / TURN / SPIN	move: single aircraft from HOVER only (Alg. 7.2), group from HOVER/FORMATION settled (Alg. 8); turn $\|°\| \le 180$ and spin also from TAKEOFF, MOVE, FORMATION, TURN	command
10	MOVE/TURN → HOVER	ARRIVED: $\|e\| < 0.06$ m, $\|v\| < 0.08$ m/s, $\|e_\psi\| < 0.05$ rad in one tick	loop
11	SPIN → HOVER	four 90° segments, each ended when $\|e_\psi\| < 0.12$ rad	loop
12	HOVER/MOVE/TURN/SPIN/FORMATION → FORMATION	all five fresh, no participant faulted, every participant z ≥ 0.70 m, safe assignment found	formation
13	FORMATION/MOVE/TAKEOFF → HOVER	proximity hold: fresh peer within 0.82 m (3D); TAKEOFF ignores parked/QUEUED peers; z ≥ 0.30 m	loop
14	same → GROUNDED	proximity hold with z < 0.30 m; disarmed, target z 0.10 m	loop
15	airborne or FAULT → LANDING	land (also clears the fault); parked aircraft are not activated	command
16	LANDING → GROUNDED	$z \le 0.20$ m and $\|v_z\| < 0.30$ m/s; disarm; LANDED	loop
17	airborne/LANDING → HOVER	hover / STOP / emergency hover; target z = max(0.25, z); parked and FAULT keep their mode	command
18	any → FAULT	fault command, exactly one named aircraft (simulation)	command
19, 20	FAULT → HOVER / GROUNDED	clear_fault: HOVER at max(0.6, z) if airborne; GROUNDED if parked or z < 0.3 m	command

5. Parameters

Quantity	Value
Loop period	0.10 s
Horizontal PD	$k_p = 0.80$, $k_d = 0.16$, saturation 0.62 m/s per axis
Vertical PD	$k_p = 1.05$, $k_d = 0.22$, saturation 0.45 m/s; landing [−0.32, +0.10] m/s
Yaw P	$k = 1.25$, saturation 0.55 rad/s
TAKEOFF → HOVER	$\|e_z\| < 0.13$ m and $\|v_z\| < 0.18$ m/s
MOVE/TURN → HOVER	$\|e\| < 0.06$ m, $\|v\| < 0.08$ m/s, $\|e_\psi\| < 0.05$ rad
Spin segment done	$\|e_\psi\| < 0.12$ rad; 4 segments of $\pm\pi/2$
Freshness	odometry age < 2.0 s

6. Complexity

O(1) per aircraft per tick; O(N) per tick overall (plus the $O(N^2)$ watchdog of Algorithm 7).

7. Properties

Guaranteed

- Commanded velocities are bounded (0.62 m/s per horizontal axis, i.e. ≤ 0.88 m/s horizontal; 0.45 m/s vertical; 0.55 rad/s yaw).
- A MOVE or TURN is reported complete only when position, speed and yaw are simultaneously within tolerance at one tick; a completed manoeuvre always returns to HOVER, so consecutive relative commands are accepted.
- No new motion is commanded to an aircraft with stale odometry; motors are not re-enabled for it.
- Stale aircraft never block a fleet-wide land/hover of the fresh aircraft; they do block (fail closed) every risk-increasing action naming them, and every formation.

Not guaranteed

- The tracking law has no integral term and no stability proof is given here; arrival may never trigger if a steady-state error ≥ 0.06 m persists (no timeout in the supervisor; Algorithm 11 adds timeouts).
- Arrival is a single-tick test (no dwell), so a transient pass through the tolerance can end a manoeuvre early.
- Per-axis saturation distorts straight-line paths (see Algorithm 5 limitations).
- FORMATION never transitions automatically; "formed" is judged externally (Algorithms 11, 15, 16).
- "Zero velocity without enable" for a stale aircraft relies on the simulator's controller behaviour; the real state of a stale aircraft is unknown.
- The yaw-segment logic of SPIN counts segments only when the error is small; a stalled segment stalls the spin.

8. Testing and measured results

- Offline: `acceptance/tests/test_missions.py::test_spin_controller`; `test_v9.py::test_move_valid_short_single`.
- Live: `acceptance/motion_test.py` (take-off, forward/back/left/right 50 cm, up/down 30 cm, turn left/right 90°, spin, land; judged from telemetry): Horus MOTIONS PASS; `flight_test.py` PASS for all five aircraft; `swarm_test.py` settle 2.13–3.64 s per leg (run `run_20261111T052914Z`). Tracking error over time is recorded by Algorithm 17; no controller-performance statistics are computed here.
- The supervisor event log (2840 lines on 2026-11-12) contains 679 `ARRIVED` events.
- Stale-skipping: no recorded live event (`SAFETY_SKIPPED` count 0 in the event log); **not measured here**.

9. Comparison with alternatives and related work

Deciding that a manoeuvre is “done” is a settling or acceptance criterion. Control engineering defines settling time as the time after which the response enters and *stays* within a band (typically 2 % or 5 % of the final value) [1], [2]. Autopilots and navigation stacks use simpler acceptance tests on position, heading and speed [3], [4], [5].

Approach	Mechanism	What it guarantees (under its assumptions)	Compared with ours
Settling time (classical) [1], [2]	Response enters and remains within a relative band around the final value.	A well-defined performance measure of the closed loop; requires “remains”, i.e. a dwell to the end of the record.	Ours uses absolute tolerances and a single tick; a transient pass through the tolerance can end the manoeuvre (no dwell).
Waypoint acceptance radius (PX4 mission mode) [3]	Switch to the next waypoint as soon as the vehicle enters a radius (NAV_ACC_RAD for multicopters).	Smooth waypoint progression; no stop required.	Ours is stricter: position, speed and yaw must all be inside tolerance, because the next relative command assumes a stable HOVER.

Goal checkers (ROS 2 Nav2) [4], [5]	SimpleGoalChecker: position and yaw tolerances (defaults 0.25 m, 0.25 rad) with an optional stateful latch on position; StoppedGoalChecker adds a velocity condition.	Configurable, documented completion semantics.	Closest to ours (position + yaw + speed), but ours has fixed, tighter tolerances (0.06 m, 0.05 rad, 0.08 m/s) and no latch.
HU-MA'AT arrival test (this document)	$\|e\| < 0.06$ m, $\|v\| < 0.08$ m/s, $\|e_\psi\| < 0.05$ rad in one tick (MOVE/TURN); $\|e_z\| < 0.13$ m, $\|v_z\| < 0.18$ m/s (TAKEOFF).	Properties of Section 7.	—

Where ours is weaker. There is no dwell or hysteresis, no timeout in the supervisor (an aircraft that never meets the tolerance stays in MOVE), FORMATION has no completion test at all, and the tolerances are not derived from the controller's measured noise. The mission coordinator compensates partly by requiring the completion condition in 3 consecutive 0.25 s samples and by per-action timeouts (Algorithm 11), which is a settling criterion in the classical sense. **Where it is adequate.** In the noise-free simulator the single-tick test has been sufficient for every recorded motion test.

10. Relation to other algorithms

Executes the setpoints produced by Algorithms 2, 5, 6, 8; runs after the watchdog (Algorithm 7) in the same tick. Its mode semantics are relied on by Algorithms 10, 11, 15, 16 and recorded by Algorithm 17.

11. References

1. K. Ogata, "Modern Control Engineering," *Prentice Hall*, 5th ed., 2010 (transient-response specifications, settling time). `ISBN 978-0-13-615673-4`
2. K. J. Åström and R. M. Murray, "Feedback Systems: An Introduction for Scientists and Engineers," *Princeton University Press*, 2008 (electronic edition by the authors). `https://www.cds.caltech.edu/~murray/amwiki/`
3. PX4 Autopilot project, "Mission Mode (multicopter): acceptance radius NAV_ACC_RAD," *PX4 User Guide*, v1.13 documentation. `https://docs.px4.io/v1.13/en/flight_modes/mission.html`
4. Nav2 project, "Simple Goal Checker," *Nav2 documentation (ROS 2)*, configuration guide, nav2_controller plugins. `https://docs.nav2.org/configuration/packages/nav2_controller-plugins/simple_goal_checker.html`
5. Nav2 project, "Stopped Goal Checker," *Nav2 documentation (ROS 2)*, configuration guide, nav2_controller plugins. `https://docs.nav2.org/configuration/packages/nav2_controller-plugins/stopped_goal_checker.html`

This specification describes the behaviour of the source revision identified above. Line-level references use `file:function`. Where the code and its own comments disagree, the code is described. Simulation-only system: no statement here is a claim about flight hardware.